

provide basic packet filtering, proxy services, and network address translation service, and they often cost only a few hundred dollars. In most cases, you simply buy the box and plug it in. At the very least, your network deserves this level of protection. If you have the time and the skill to build a firewall, you can use a firewall package or the firewall tools built into your operating systems. A firewall package increases initial cost, but it is easy to work with. The packet filtering tools built into the operating system cost nothing but are the most difficult to configure. The `iptables` tool provided with Linux is a good example of the type of firewall tools provided with some Unix operating systems.

Filtering Traffic with `iptables`

In its simplest incarnation, a firewall is a filtering router that screens out unwanted traffic. Use the routing capabilities of a multi-homed Linux host combined with the filtering features of `iptables` to create a filtering router.

The Linux kernel categorizes firewall traffic into three groups and applies different filter rules to each category of traffic. These are:

INPUT

Incoming traffic bound for a process on the local system is tested against the INPUT filter rules before it is accepted.

OUTPUT

Outbound traffic that initiated on the local system is tested against the OUTPUT filter rules before it is sent.

FORWARD

Traffic from one external system bound for another external system is tested against the FORWARDING filter rules.

The INPUT and OUTPUT rules are used when the system acts as a host. The FORWARD rules are used when the system acts as a router. In addition to the three standard categories, `iptables` accepts user-defined categories.

Defining `iptables` filter rules

The Linux kernel maintains a list of rules for each of these categories. The lists of rules are maintained by the `iptables` command.* Use the options shown in Table 12-2 with the `iptables` command to create or delete user-defined chains, to add rules to a chain, to delete rules from a chain, and to change the order of the rules in the chain.

* `iptables` came into use with Linux kernel 2.4. Early kernels used the `ipfwadm` and the `ipchains` commands. See *Linux Firewalls* by Robert Ziegler (New Riders, 2000) for information on these older commands.

Table 12-2. *iptables* command-line options

Option	Function
-A	Appends rules to the end of a ruleset.
-D	Deletes rules from a ruleset.
-E	Renames a ruleset.
-F	Removes all of the rules from a ruleset.
-I	Inserts a rule into a specific location in a ruleset.
-L	Lists all rules in a ruleset.
-N	Creates a user-defined ruleset with the specified name.
-P	Sets the default policy for a chain.
-R	Replaces a rule in a chain.
-X	Deletes the specified user-defined ruleset.
-Z	Resets all packet and byte counters to zero.

Firewall rules are composed of a filter against which the packets are matched and the action taken when a packet matches the filter. The action can either be a standard policy or a jump to a user-defined ruleset for additional processing. The `-j target` command-line option identifies the user-defined ruleset or the standard policy to handle the packet. *target* is either the name of a ruleset or a keyword that identifies a standard policy. The keywords for the standard policies are:

ACCEPT

Let the packet pass through the firewall.

DROP

Discard the packet.

QUEUE

Pass the packet up to user space for processing.

RETURN

In a user-defined ruleset, this means to return to the ruleset that called this ruleset. In one of the three kernel rulesets, this means to exit the chain and use the default policy for the chain.

The *iptables* command constructs filters that match on the protocol used, the source or destination address, or the network interface used for the packet, using a variety of command-line parameters. The basic *iptables* parameters for building filters are:

`-p protocol`

Defines the protocol to which the rule applies. *protocol* can be any numeric value from the `/etc/protocols` file or one of the keywords: `tcp`, `udp`, or `icmp`.

- s *address[/mask]*
Defines source address of the packets to which the rule applies. *address* can be a hostname, network name, or IP address.
- sport [*port[:port]*]
Defines the source port of the packets to which the rule applies. *port* can be a name or number from the */etc/services* file. A range of ports can be specified using the format *port:port*. If no specific port value is specified, all ports are assumed.
- d *address[/mask]*
Defines the destination address of the packets to which the rule applies. *address* can be a hostname, network name, or IP address.
- dport [*port[:port]*]
Defines the destination port to which the rule applies. This filters all traffic bound for a specific port. The port is defined using the same rules as those used to define these values for the packet source.
- icmp-type *type*
Defines the ICMP type to which the rule applies. *type* can be any valid ICMP message type number or name.
- i *name*
Defines the name of the input network interface to which the rule applies. Only packets received on this interface are affected by the rule. Specify a partial interface name by ending it with a + (e.g., *eth+* matches all Ethernet interfaces that begin with *eth*).
- o *name*
Defines the name of the output network interface to which the rule applies. Only packets sent out this interface are affected by the rule. Specify a partial interface name by ending it with a + (e.g., *eth+* matches all Ethernet interfaces that begin with *eth*).
- f
Indicates that the rule refers only to second and subsequent fragments of fragmented packets.

Sample iptables commands

Putting this all together creates a firewall that can protect your network. Assume we have a Linux router attached to a perimeter network with the address 172.16.12.254 on interface *eth0* and to an external network with the address 192.168.6.5 on interface *eth1*. Further assume that the perimeter network contains only a sendmail server and an Apache server. Here is an example of some iptables commands we might use on the Linux system to protect the perimeter network:


```

iptables -F INPUT
iptables -F FORWARD
iptables -A INPUT -i eth1 -j DROP
iptables -A FORWARD -i eth1 -s 172.16.0.0/16 -j DROP
iptables -A FORWARD -o eth1 -d 172.16.0.0/16 -j DROP
iptables -A FORWARD -d 172.16.12.1 25 -j ACCEPT
iptables -A FORWARD -d 172.16.12.6 80 -j ACCEPT
iptables -A FORWARD -j DROP

```

The first two commands use the -F option to clear the rulesets we plan to work with. The third line drops any packets from the external network that are bound for a process running locally on the Linux router. We do not allow any access to router processes from the external world.

The next two commands drop packets that are being routed to the external world using an internal address. If packets are received on the external interface with a source address from the internal network, they are dropped. Likewise, if packets are being sent out the external interface with a destination address from the internal network, they are dropped. These rules say that if packets on the external network interface (eth1) misuse addresses from the internal network (172.16), somebody is trying to spoof us and the packets should be discarded.

The next two rules are basically identical. They accept packets if the destination and port are the correct destination and port for a specific server. For example, port 25 is the SMTP port and 172.16.12.1 is the mail server, and port 80 is the HTTP port and 172.16.12.6 is the web server. We accept these inbound connections because they are destined for the correct systems. The last rule rejects all other traffic.

These examples illustrate the power of Linux's built-in filtering features and provide enough information to get you started. Clearly much more can and should be done to build a real firewall. If you want to know more about iptables, see *Building Internet Firewalls* and *Linux Security*, both mentioned in the reading list below, for many more detailed examples.

Words to the Wise

I am not a security expert; I am a network administrator. In my view, good security is good system administration and vice versa. Most of this chapter is just common-sense advice. It is probably sufficient for most circumstances, but certainly not for all.

Make sure you know whether there is an existing security policy that applies to your network or system. If there are policies, regulations, or laws governing your situation, make sure to obey them. Never do anything to undermine the security system established for your site.

No system is completely secure. No matter what you do, you will have problems. Realize this and prepare for it. Prepare a disaster recovery plan and do everything